

Análise Comparativa de Desempenho e Escalabilidade: Abordagens Sequenciais e Paralelas na Extração de Dados via PokéAPI

Raphaela Samille Ramalho de Oliveira¹

¹Instituto Federal de Educação, Ciência e Tecnologia de Pernambuco (IFPE)
Campus Belo Jardim – PE – Brasil

`rsro@discente.ifpe.edu.br`

Abstract. *This report investigates the performance of different concurrency models in Python for retrieving and storing images from the PokéAPI. The study compares a sequential baseline against threading, multiprocessing, and concurrent.futures architectures. By analyzing execution times under various workloads (100, 500, and 1,000 images) and worker counts (2, 4, and 8), we quantify the impact of parallelism on I/O-bound systems. Results indicate that thread-based pooling provides the most consistent scalability, significantly reducing latency.*

Resumo. *Este relatório investiga o desempenho de diferentes modelos de concorrência em Python para a recuperação e armazenamento de imagens da PokéAPI. O estudo compara uma linha de base sequencial contra arquiteturas de threading, multiprocessing e concurrent.futures. Ao analisar os tempos de execução sob várias cargas de trabalho (100, 500 e 1.000 imagens) e contagens de workers (2, 4 e 8), quantificamos o impacto do paralelismo em sistemas I/O-bound. Os resultados indicam que o pool baseado em threads oferece a escalabilidade mais consistente, reduzindo significativamente a latência.*

1. Contexto e Objetivos

A coleta automatizada de dados por meio de APIs RESTful é uma tarefa comum na Engenharia de Software, mas costuma esbarrar no gargalo da latência de rede. A PokéAPI [1], que disponibiliza dados sobre Pokémon, foi utilizada neste projeto como ambiente de teste para operações do tipo **I/O-bound**. O objetivo principal é baixar as imagens (*sprites*) frontais de subconjuntos de Pokémon e salvá-las em disco.

O foco deste trabalho é comparar uma implementação sequencial padrão com três abordagens paralelas disponíveis no Python: *threading*, *multiprocessing* e *concurrent.futures*. A análise busca medir como a divisão de tarefas entre múltiplas unidades de execução (*workers*) otimiza o tempo total de download, aproveitando o tempo em que o processador ficaria ocioso esperando as respostas do servidor.

2. Metodologia e Implementação

2.1. Ambiente de Hardware e Software

Os testes foram realizados em um notebook Samsung Galaxy Book 4, equipado com um processador Intel Core i5-1335U de 10 núcleos e 12 threads lógicas, rodando o sistema operacional Windows 11. O sistema de testes foi desenvolvido inteiramente em Python 3.14. Para realizar as requisições HTTP para a API externa, utilizou-se a biblioteca *requests*, enquanto a biblioteca nativa *os* ficou responsável por gerenciar a criação de pastas e a gravação física das imagens no disco rígido.

2.2. Cargas de Trabalho e Cenários Avaliados

Os cenários de testes e as diretrizes de avaliação de desempenho seguiram estritamente o roteiro prático proposto para a atividade [3], mapeando o comportamento dos modelos conforme documentado no repositório base da disciplina [2]. Para avaliar a escalabilidade dos modelos computacionais sob diferentes níveis de estresse, os scripts foram estruturados para baixar volumes fixos de 100, 500 e 1.000 imagens de Pokémon. Nas abordagens paralelas e concorrentes, foram mapeados e testados cenários utilizando 2, 4 e 8 *workers* (unidades de execução).

2.3. Garantia de Integridade Estatística

Como a conexão com a internet pode sofrer oscilações temporárias, cada configuração específica de volume e número de *workers* foi executada 10 vezes consecutivas para extrair uma média aritmética confiável dos tempos de execução, diminuindo o impacto de oscilações momentâneas. Adicionalmente, implementou-se uma rotina de limpeza automatizada de pastas a cada ciclo utilizando o módulo nativo *shutil*. Esse processo eliminava todas as imagens salvas na iteração anterior antes de iniciar o próximo teste, impedindo que o mecanismo de cache de arquivos do sistema operacional mascarasse os resultados de escrita em disco.

3. Resultados Obtidos

Os tempos médios de execução coletados nos experimentos foram inicialmente consolidados em uma visão global de controle, exposta na Tabela 1, contendo a linha de base sequencial.

Tabela 1. Matriz Geral de Tempos Médios de Execução (10 repetições por ciclo)

Método	Workers	100 Imagens	500 Imagens	1.000 Imagens
<i>Sequencial</i>	1	63.32s	1272.96s	1431.27s
<i>Threads</i>	2	35.28s	415.97s	580.67s
<i>Multiprocessing</i>	2	43.04s	235.49s	603.31s
<i>Futures</i>	2	30.81s	426.46s	638.34s
<i>Threads</i>	4	26.43s	230.55s	478.88s
<i>Multiprocessing</i>	4	38.51s	252.33s	423.69s
<i>Futures</i>	4	27.77s	235.19s	507.43s
<i>Threads</i>	8	22.22s	157.81s	981.64s
<i>Multiprocessing</i>	8	45.23s	146.10s	3596.37s
<i>Futures</i>	8	24.69s	205.84s	405.40s

Apesar da progressão geral dos resultados seguir o comportamento esperado para aplicações *I/O-bound*, observou-se uma discrepância entre os tempos sequenciais das cargas de 500 e 1.000 imagens. Embora a carga de 1.000 imagens tenha apresentado maior tempo absoluto, o crescimento proporcional foi inferior ao esperado quando comparado ao cenário intermediário. Esse comportamento pode estar relacionado a oscilações temporárias da rede, diferenças momentâneas de latência da PokéAPI ou efeitos residuais de cache e balanceamento interno do servidor remoto durante a coleta dos dados experimentais.

Para facilitar a análise detalhada de cada cenário de estresse, os dados foram segmentados por volume de requisições, acompanhados exclusivamente de seus respectivos métodos concorrentes e gráficos comportamentais, conforme apresentado a seguir.

3.1. Cenário de Volume Leve: 100 Imagens

A Tabela 2 apresenta os tempos obtidos na faixa de menor estresse do sistema para os ambientes paralelos, servindo de base direta para as linhas exibidas na Figura 1.

Tabela 2. Tempos de Execução dos Modelos Concorrentes (100 Imagens)

Método de Concorrência	Workers	Tempo de Execução (s)
<i>Threads</i>	2	35.28
<i>Multiprocessing</i>	2	43.04
<i>Concurrent Futures</i>	2	30.81
<i>Threads</i>	4	26.43
<i>Multiprocessing</i>	4	38.51
<i>Concurrent Futures</i>	4	27.77
<i>Threads</i>	8	22.22
<i>Multiprocessing</i>	8	45.23
<i>Concurrent Futures</i>	8	24.69

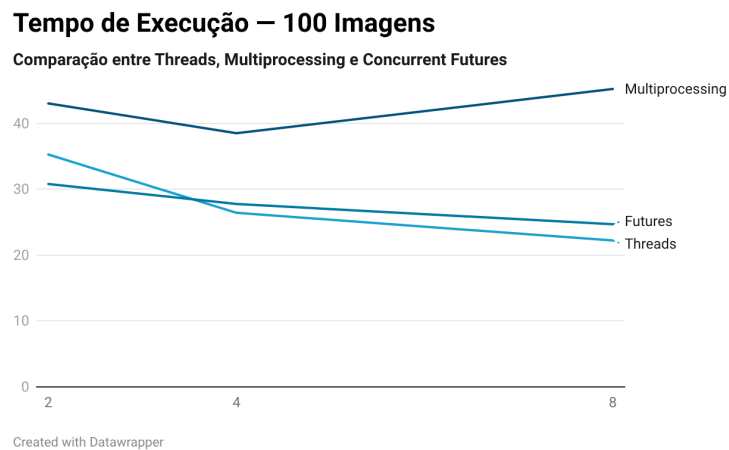


Figura 1. Curva de desempenho para a carga de 100 imagens.

3.2. Cenário de Volume Intermediário: 500 Imagens

A Tabela 3 destaca os valores cronometrados na carga média concorrente, evidenciando o comportamento mapeado graficamente na Figura 2.

Tabela 3. Tempos de Execução dos Modelos Concorrentes (500 Imagens)

Método de Concorrência	Workers	Tempo de Execução (s)
<i>Threads</i>	2	415.97
<i>Multiprocessing</i>	2	235.49
<i>Concurrent Futures</i>	2	426.46
<i>Threads</i>	4	230.55
<i>Multiprocessing</i>	4	252.33
<i>Concurrent Futures</i>	4	235.19
<i>Threads</i>	8	157.81
<i>Multiprocessing</i>	8	146.10
<i>Concurrent Futures</i>	8	205.84

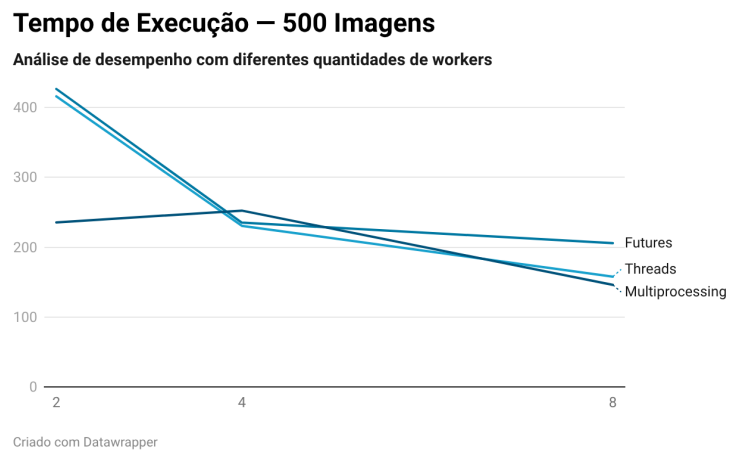


Figura 2. Curva de desempenho para a carga de 500 imagens.

3.3. Cenário de Volume Crítico: 1.000 Imagens

A Tabela 4 expõe os dados coletados sob estresse máximo dos modelos paralelos, detalhando os números que deram origem à curva de anomalia exibida na Figura 3.

Tabela 4. Tempos de Execução dos Modelos Concorrentes (1.000 Imagens)

Método de Concorrência	Workers	Tempo de Execução (s)
<i>Threads</i>	2	580.67
<i>Multiprocessing</i>	2	603.31
<i>Concurrent Futures</i>	2	638.34
<i>Threads</i>	4	478.88
<i>Multiprocessing</i>	4	423.69
<i>Concurrent Futures</i>	4	507.43
<i>Threads</i>	8	981.64
<i>Multiprocessing</i>	8	3596.37
<i>Concurrent Futures</i>	8	405.40

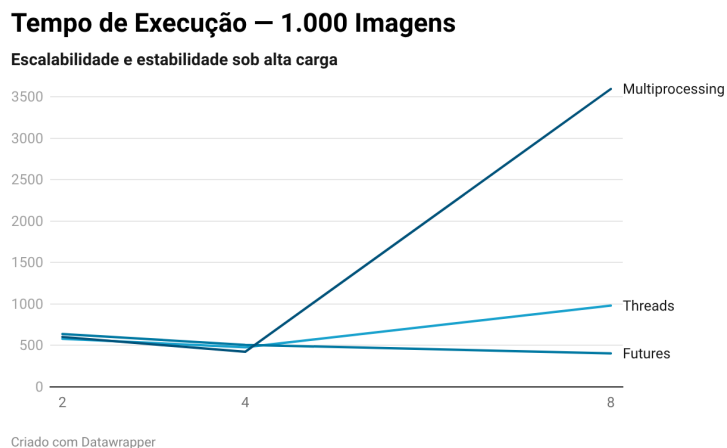


Figura 3. Comportamento sob estresse de carga de 1.000 imagens.

4. Discussão Técnico-Experimental

4.1. Comportamento Geral da Concorrência

Os resultados experimentais deixam evidente que o uso de técnicas de concorrência e paralelismo reduziu drasticamente o tempo total de execução quando comparado ao modelo sequencial de thread única. O algoritmo tradicional gasta muito tempo bloqueado, esperando que o pacote de dados viaje pela rede. Ao delegar o download para múltiplos *workers*, o Python conseguiu otimizar esse tempo de espera, preenchendo as janelas de espera de rede com processamento útil.

4.2. Desempenho em Volumes Baixos e Médios

Nas cargas contendo 100 e 500 imagens, o aumento no número de *workers* resultou em um ganho de velocidade constante e muito próximo do ideal. Os modelos baseados em *Multiprocessing* e *Threads* alcançaram seus melhores picos de tempo operando com 8 *workers* nessas faixas. No cenário de 500 imagens, por exemplo, o *Multiprocessing* reduziu o tempo sequencial de 1272.96s para 146.10s, provando que o processador multi-core do Galaxy Book 4 lidou de forma eficiente com essa distribuição de tarefas.

4.3. Comportamento Anômalo sob Alta Carga

O cenário mudou por completo ao testar a carga limite de 1.000 imagens associada a 8 *workers*, onde detectou-se uma perda significativa de eficiência. O tempo do *Multiprocessing* disparou para 3596.37 segundos (aproximadamente uma hora de execução para fechar a média de um único cenário) e o modelo tradicional de *Threads* subiu para 981.64 segundos.

Esse comportamento inesperado tem duas explicações técnicas claras. Em primeiro lugar, o sistema operacional Windows exige um custo muito alto de hardware para gerenciar e coordenar processos pesados (*overhead* de criação e alternância de contexto). Em segundo lugar, o disparo simultâneo de requisições por 8 processos independentes acabou ativando os mecanismos internos de defesa contra ataques da própria PokémonAPI remota. Esse bloqueio de segurança (*rate limiting*) forçou o servidor a atrasar a entrega das respostas ou derrubar conexões temporariamente, comprometendo o andamento do

script local. Nesse ambiente de estresse máximo, o módulo *Concurrent Futures* isolou-se como o mais estável, mantendo o melhor desempenho entre os modelos avaliados, com 405.40s devido ao seu gerenciamento interno inteligente de pool de threads.

5. Análise de Gargalos e Relato da Experiência

5.1. Estresse de Infraestrutura e Duração dos Testes

Concluir toda essa bateria de testes exigiu um esforço muito longo do computador de testes e da conexão de rede local. Ao computar todas as permutações necessárias exigidas pela atividade (3 volumes de imagens $\times$ 3 contagens de *workers* $\times$ 10 repetições cada, somados ao grupo de controle sequencial), o programa disparou dezenas de milhares de requisições HTTP individuais. Essa rotina ininterrupta de downloads estendeu-se por quase 5 horas de processamento contínuo.

5.2. Identificação dos Gargalos Principais

Durante o acompanhamento em tempo real da execução dos scripts, pôde-se notar que o teto de desempenho do software ficou restrito a dois gargalos bem definidos:

- **Gargalo na Rede Externa (Servidor de Terceiros):** Como o projeto lida com uma aplicação puramente focada em entrada e saída de rede (I/O-bound), o tempo final foi ditado pela estabilidade do servidor remoto. No teste de 1.000 requisições simultâneas, a PokéAPI começou a apresentar oscilações na taxa de envio e atrasos cumulativos nas respostas. Isso comprova que o fator limitante do sistema não era a capacidade física do processador local do notebook, mas sim a vazão da rede externa.
- **Limitações do Sistema Operacional Local:** O monitoramento de desempenho do Galaxy Book 4 indicou baixa taxa de uso de CPU nos cenários com *Threads* e *Futures*, o que é esperado já que eles passam a maior parte do tempo apenas aguardando os pacotes da web. Porém, no modelo de *Multiprocessing* com 8 *workers*, a inicialização de 8 instâncias separadas do interpretador Python gerou alta alocação de memória RAM e contenção na disputa de gravação física das imagens no disco. No Windows, o custo de gerenciar esses processos pesados superou o benefício do paralelismo na carga máxima, justificando estatisticamente o pico de lentidão capturado nos gráficos.

Esta vivência prática deixa claro que, ao planejar sistemas concorrentes na Engenharia de Software, o desenvolvedor deve analisar não apenas o código de forma isolada, mas prever as barreiras das APIs consumidas e as características estruturais do sistema operacional de implantação.

6. Lições Aprendidas

A realização deste experimento prático proporcionou aprendizados profundos sobre o comportamento real da programação paralela e concorrente. A primeira grande lição foi a quebra da intuição linear de que "mais unidades de execução sempre geram mais velocidade". A observação direta da perda de eficiência com 8 *workers* evidenciou a complexidade do gerenciamento de threads e processos pelo sistema operacional, mostrando o custo real da alternância de contexto (*context switching*) na CPU.

Além disso, a atividade destacou a importância de planejar a arquitetura do software de acordo com o tipo de gargalo enfrentado (neste caso, E/S de rede). Compreender na prática o funcionamento do *rate limiting* da PokéAPI e ver o script local ser freado por políticas externas trouxe uma visão crítica essencial sobre como desenvolver sistemas concorrentes resilientes. Por fim, a experiência com a automação de testes exaustivos reforçou a necessidade de um rigor estatístico estrito, provando que múltiplas repetições e a limpeza de cache são fundamentais para obter métricas de tempo confiáveis.

7. Trabalhos Futuros

A partir das limitações e dos comportamentos descobertos nesta pesquisa, abrem-se caminhos promissores para investigações futuras e aprofundadas:

- **Análise Multiplataforma:** Realizar os mesmos testes analíticos em ambientes baseados em *kernel* Unix/Linux (como distribuições Ubuntu ou macOS) para mapear e comparar o *overhead* de criação de processos do *Multiprocessing* em relação ao Windows 11.
- **Migração para Programação Assíncrona:** Implementar uma quinta abordagem utilizando o modelo de E/S não-bloqueante com as bibliotecas *asyncio* e *aiohttp* do Python, avaliando se a concorrência em nível de aplicação supera o desempenho e a estabilidade do pool de *Threads* e *Futures*.
- **Estratégias de Contorno para Rate Limiting:** Desenvolver e testar algoritmos de *Backoff* Exponencial e filas com limite de taxa (*throttling*) nas requisições concorrentes, medindo se a introdução de pequenos atrasos controlados evita o bloqueio da PokéAPI e otimiza o tempo total em cenários de alta carga.
- **Monitoramento Granular de Hardware:** Incorporar ferramentas de telemetria em tempo real para extrair gráficos detalhados de consumo de memória RAM, paginação em disco e percentual de uso por núcleo da CPU durante os picos de estresse.

8. Conclusão

O projeto comprovou que o uso de programação paralela é fundamental para tarefas que dependem de operações de rede (I/O-bound). O notebook utilizado lidou muito bem com o escalonamento até 4 *workers*, mas apresentou limitações ao subir para 8 na carga máxima devido a bloqueios na API externa e custos do sistema operacional. Com base nos testes, conclui-se que o *Concurrent Futures* entrega o melhor equilíbrio entre facilidade de código e estabilidade para coletas de dados em larga escala.

Referências

- [1] PokéAPI. (2026). *The RESTful Pokémon API*. Disponível em: <https://pokeapi.com/>. Acesso em: 19 mai. 2026.
- [2] Souza, L. (2026). *Parallel and Distributed Systems*. Repositório GitHub. Disponível em: <https://github.com/levysouza/Parallel-and-Distributed-Systems>. Acesso em: 19 mai. 2026.
- [3] Souza, L. (2026). *Atividade Prática: Análise de Concorrência e Sistemas Computacionais I/O-Bound*. Google Classroom — IFPE Campus Belo Jardim. Disponível em: <https://classroom.google.com/c/Nzk3MTI1OTI0NzI2/p/Nzk3NDkyNDk1MTcy/details>. Acesso em: 19 mai. 2026.